

CSCI203 ASSIGNMENT TWO

It is worth **17.5%**

Due 17:00 Friday 23/10/2026 & must be submitted via the subject Moodle

Task:

This assignment involves an extension to the single source-single destination shortest/longest path problem. You are required to implement **TWO different shortest path algorithms**:

- Dijkstra's Algorithm
- A* Algorithm

and **ONE longest path algorithm**:

- DFS Algorithm

Your program should:

1. Read the name of a text file – “*a2-sample.txt*” from the console, which means that you should get file name from users' input.
2. Read a directed weighted graph from the file.
3. Find the shortest path between the start and the goal vertices specified in the file.
4. Print out the vertices on the shortest path, in order from the start to the goal.
5. Print out the length of the shortest path.
6. Print out the number of expanded nodes (The total number of nodes actually processed by the algorithm, including those that end up not on the shortest path).
7. Find the longest path between the start and the goal vertices.
8. Print out the vertices on the longest path, in order from the start to the goal (each vertex can only be visited once maximally).
9. Print out the length of the longest path.

Note: You are required to implement two shortest path algorithms (Dijkstra and A*) and one longest path algorithm (DFS) in a **SINGLE** program file. Each algorithm **MUST** be executed **separately**, and their results **MUST** be printed **individually**. The output format should follow the style shown in **Figure 1**.

Input:

The data files are constructed as follows:

- The sample file – “*a2-sample.txt*” is provided to describe the graph.
- **First line:** Contains two integers – “*nVertices*” and “*nEdges*”, which are the number of vertices and edges in the graph.
- **Next “*nVertices*” lines:** Vertices triples - “*k*”, “*x*”, “*y*” (an int followed by two doubles),
 - “*k*” is a unique vertex label (not guaranteed consecutive).
 - “*x*”, “*y*” are coordinates of the vertex.
- **Next “*nEdges*” lines:** Edges triples – “*i*”, “*j*”, “*w*” (Two ints followed by a double),
 - “*i*” and “*j*” are directed edge $i \rightarrow j$
 - “*w*” is the weight of the edge

Note: $w \geq 0$ and $w \geq$ the Euclidean distance; no self-loops; multi-edges allowed.

- **Final Line:** “Start” and “Goal” vertices (Two ints).
- **Sample file:** In “*a2-sample.txt*”, the first line is “20” and “100”, followed by 20 vertex lines, 100 edge lines, and a last line “2” and “13”.
- You **CANNOT** use hard-coding parsing with absolute line numbers (e.g., “ends on line 121”). The number of vertices and edges must be read from the “*nVertices/nEdges*” line.

- Your code should also validate the data format, such as checking the legality of numeric values and ensuring the correct formatting.

Output:

Output of the program will consist of the following data:

- The **number of vertices** and **edges** in the graph
- The **start** and the **goal vertices**
- The **Euclidean distance** between the **start** and the **goal vertices** is calculated using their coordinates
- The **vertices** on the **shortest path**, in order from the start to the goal.
- The **weight** of the **shortest path**.
- The **number of expanded nodes**.
- The **vertices** on the **longest path** (each vertex can only be visited once maximally), in order from the start to the goal.
- The **weight** of the **longest path**.

The Example print can be seen as follow (exact prefixes; numeric values rounded to **three decimals**):

```
Vertices: <nVertices>, Edges: <nEdges>
Start: <start>, Goal: <goal>
Euclidean distance (start-goal): <dist with 3 decimals>
##### Shortest path by Dijkstra's Algorithm: #####
Shortest path: <v1 v2 ... vk>      # or No path
Shortest length: <length with 3 decimals> # or INF
Number of expanded nodes: <int>|
##### Shortest path by A* Algorithm #####
Shortest path: <v1 v2 ... vk>      # or No path
Shortest length: <length with 3 decimals> # or INF
Number of expanded nodes: <int>
##### Longest Path by DFS #####
Longest path: <v1 v2 ... vm>      # or No path
Longest length: <length with 3 decimals> # or INF
```

Figure1. Example of output format.

- If there are **multiple optimal paths** (same length), output the **lexicographically smallest vertex sequence** (compare labels as numbers left-to-right).
- **Numeric tolerance** for judging: ± 0.001 on distances/lengths; always **print** with exactly **3 decimals**.
- The data **does not guarantee** that start $\rightarrow$ goal is reachable. If unreachable:
 - **Shortest path**: print No path and INF;

- **Longest path:** print No path and INF.

Requirement:

1. Purpose & Self-Implementation

- This assignment is designed to deepen your understanding of data structures and algorithms.
- You must **implement all core data structures and algorithms yourself** (no third-party data structure/algorithm libraries).

2. Academic Integrity & Citations

- You may use any techniques covered in class **without citation**.
- If you consult materials beyond the lecture notes (blogs, papers, textbooks, code snippets, etc.), you **must cite** them with a clear in-code comment right where they are used (URL or full reference).
- Any copied code (beyond trivial boilerplate) **earns zero marks for that portion**, even if cited; uncited copying is plagiarism and may incur further penalties.

3. Allowed Libraries (I/O & Basic Math Only)

Use only standard I/O and basic math utilities. No data-structure/algorithm libraries beyond what is listed below.

- **C++:** Allowed: `iostream`, `fstream`, `cmath`.
Disallowed: all STL containers/algorithms (`vector`, `queue`, `priority_queue`, etc.) and any third-party libraries.
- **Java:** Allowed: `java.io.*`, `java.lang.Math`.
Disallowed: `java.util.*` collections and any third-party libraries.
- **Python:** Allowed: built-ins/modules such as `list`, `dict`, `heapq`, `math`, `sys`, `io`.
Disallowed: third-party packages and graph/algorithm libraries (e.g., `networkx`, `numpy`, etc.).

Note: You may use language **built-ins** (e.g., Python lists/dicts, `heapq`) for basic containers/priority queues, but the **algorithmic logic** (e.g., Dijkstra, DFS/branch-and-bound, Dynamic Programming) must be your own implementation.

4. Code Quality

- Your code must be **readable** and **adequately commented**: explain key data structures, the algorithmic approach, and any non-obvious logic.
- Comments for external references (if any) must be placed **at the point of use**.

Report:

A pdf file describing your solution and program output should be produced. This file should contain:

1. A high-level description (with pseudo-code) of the overall solution strategy in **all three algorithms**. Additionally, the description should include:
 - a. **Dijkstra's Algorithm:** Explain how you maintain the priority queue and distance/parent arrays.
 - b. **A* Algorithm:** Provide the evaluation function $f(v) = g(v) + h(v)$. Then, define the heuristic function and explain why it is admissible and consistent.

- c. **DFS Algorithm:** Describe your search/pruning strategy, and analyze complexity and report observed performance.
2. A complexity analysis of your solution with big-O notation and sufficient justification in **all three algorithms**. Compare with the results of Dijkstra and A*, what's the difference? Then, conclude when and why A* outperforms Dijkstra.
3. A list of all the data structures used, and the reasons for using them.
4. A snapshot of the compilation and the execution of your program on the provided "a2-sample.txt" file.
5. The outputs (the shortest and longest paths) are produced by your program on the provided "a2-sample.txt" file.
6. The report pdf file should be called <student number>-a2.pdf.

Submission:

- Please submit your **ONE** source code file and the **ONE** pdf report as a zip file (named as <student number used language>-a2.zip, e.g. 123456Python-a2.zip, 123456Java-a2.zip, 123456CPP-a2.zip) to CSCI203 Moodle site (Assignment 2 submission folder) before the deadline.
- Please note that the email submission will NOT be accepted.
- If an extension is required, please submit an academic consideration via SOLS before the deadline.

NOTE: Name your files strictly according to the above naming rules. Because we may need to batch compile your code files, the wrong name may cause the compilation to fail.

Marking Guide:

- **Code** is worth **70%** of the total mark for this assignment, and **report** is worth **30%** of the total mark for this assignment. The following cases will be directly labelled with a **mark of 0**:
 - A program that fails to compile or run.
 - If your program produces different output from what is reported in the pdf file.
 - Plagiarism, copy the whole code directly, use Generative AI directly to generate code.
- Failure to name according to the rules will **deduct 30% of the total mark**.
- Programs that lack clarity, both in code and comments, will lose marks. The total mark will be determined based on both your code and the accompanying design pdf document. Detailed as follow:

Code (70% of total mark):

Overall: If the output is wrong but student shows serious attempt, up to 80% of the total mark of code part deducted. No comments will deduct 50% of the total mark of code part.

- **A* (25%)**
 - Correctness output (15%)
 - Produces the correct shortest path and distance, length.
 - Expands nodes properly and terminates at goal.
 - Implementation quality (5%)
 - Correctly maintains $f(v)=g(v)+h(v)$ with Euclidean heuristic.
 - Proper use of data structure.
 - Clarity (5%)
 - Clear variable names
 - Comments explain key steps

- **Dijkstra (25% of code part)**
 - Correctness output (15%)
 - Produces the correct shortest path and distance, length.
 - Expands nodes properly and terminates at goal.
 - Implementation quality (5%)
 - Proper use of data structure.
 - Distance and parent arrays updated consistently.
 - Clarity (5%)
 - Clear variable names
 - Comments explain key steps
- **DFS (20% of code part)**
 - Correctness output (10%)
 - Produces the correct longest path and distance, length.
 - Implementation quality (5%)
 - Proper use of data structure.
 - Produces a valid longest simple path (no repeated vertices).
 - Clarity (5%)
 - Clear variable names
 - Comments explain key steps

Report (30% of total mark)

Overall: The report **MUST NOT** exceed **10** pages. Exceeding this limit will result in a 10% penalty. All screenshots included in the report **MUST** be clear and readable. Unclear or blurry screenshots will result in a 10% penalty.

- **Algorithm Descriptions (13%)**
 - Overall solution strategy + pseudo-code for all three algorithms (5%)
 - Dijkstra: explain distance/parent arrays (3%)
 - A*: explain evaluation function $f(v)=g(v)+h(v)$, define heuristic, and justify admissibility/consistency (3%)
 - DFS: describe search strategy, discuss complexity, and report observed performance (2%)
- **Complexity Analysis (12%)**
 - Big-O analysis for all three algorithms (4%)
 - Comparison: Dijkstra vs. A* (4%)
 - Explain when and why A* may outperform Dijkstra (4%)
- **Data Structures Justification (5%)**
 - List the data structures used.
 - Justify why each was chosen (e.g., heap for Priority Queue, arrays for visited, etc.).